Model and Field Validators

Good call—**model validators** are where most real-world logic lives, and where subtle bugs usually hide.

Let's go deep, but in a way that builds *intuition* + *debugging ability*, not just syntax.

🔹 Core Idea (Anchor This)

A **model validator** sees the *whole picture*.

- `mode="before"` → you get the **raw input (dict)**
- `mode="after"` → you get the **fully built model instance**

Everything else is just variations of this.

🔸 1. `mode="before"` — Input Shaping Layer

Think of this as a **data cleaning / normalization hook**.

```python
from pydantic import BaseModel  model_validator

class User BaseModel
    name  str

    @model_validator mode="before"
    @classmethod
    def clean_input cls  data
        if "name" in data
            data "name"  = data "name" .         .
        return data
```

🔍 What you actually get:

- `data` is a **dict**
- No types enforced yet
- You can:
  - add/remove keys
  - transform raw values

⚠ Critical Pitfall

```python
data "age" + 1 # ❌ might be string
```

👉 Because parsing hasn't happened yet.

✅ When to use `before`

- messy API input
- multiple input formats
- alias handling
- default injection

🔸 Advanced Pattern: Flexible Input

```python
class User BaseModel
    name str

    @model_validator mode="before"
    @classmethod
    def handle_multiple_formats cls data
        if isinstance data str
            return "name" data
        return data
```

👉 Now this works:

```python
User "john"
User "name" "john"
```

🔸 2. `mode="after"` — Business Logic Layer

This is where **serious validation happens**.

```python
class Event BaseModel
    start int
```

```
    end  int

    @model_validator mode="after"
    def validate_range self
        if self.        >= self.
            raise ValueError "start must be less than end"
        return self
```

🔍 What you get:

- `self` = fully validated model
- All fields exist (unless optional)

👉 This is why experts prefer this mode.

🔸 Mutation Allowed (Important)

Python

```
@model_validator mode="after"
def compute self
    self.            = self.       - self.
    return self
```

✔️ You can modify the model
✔️ Useful for derived fields

⚠️ Pitfall: Forgetting `return self`

Python

```
def validate self
    if bad
        raise ValueError
    # ❌ forgot return
```

👉 This breaks silently in weird ways

🔥 Execution Order (Deep Understanding)

[redacted]

🔸 3. Chaining Multiple Model Validators

Yes, you can have multiple:

```python
@model_validator mode="before"
def step1 cls  data

@model_validator mode="before"
def step2 cls  data
```

👉 They run **top to bottom**

Same for `after`.

🧠 Expert Insight

Split logic like:

- Validator 1 → normalize
- Validator 2 → enforce rules

👉 makes debugging MUCH easier

🔸 4. Real-World Patterns

✅ Pattern 1: Conditional Required Fields

```python
class Payment BaseModel
    method  str
```

```
    card_number  str | None = None

    @model_validator mode="after"
    def check self
        if self.            == "card" and not self.
            raise ValueError "card_number required"
        return self
```

✅ Pattern 2: Mutual Exclusivity

```python
class Query BaseModel
    id  int | None = None
    name  str | None = None

    @model_validator mode="after"
    def only_one self
        if self.    and self.
            raise ValueError "Provide either id or name, not both"
        return self
```

✅ Pattern 3: Derived Fields

```python
class Order BaseModel
    price  float
    quantity  int
    total  float = 0

    @model_validator mode="after"
    def compute self
        self.        = self.        * self.
        return self
```

✅ Pattern 4: Data Migration Layer

```python
@model_validator mode="before"
def migrate cls  data
    if "full_name" in data
```

```
    data "name"  = data.      "full_name"
    return data
```

👉 Extremely common in evolving APIs

🔸 5. Debugging Model Validators (Real Skill)

🧩 Problem: “Why is my validator not working?”

Checklist:

1. Wrong mode?
   - using `before` but expecting parsed values
2. Forgot return?
   - must return `data` or `self`
3. Exception type?
   - use `ValueError`, not random exceptions
4. Validator not triggered?
   - input shape mismatch

🧩 Problem: Unexpected values

Add temporary debug:

Python

```python
@model_validator mode="before"
def debug cls  data
    print "RAW:"  data
    return data
```

Python

```python
@model_validator mode="after"
def debug self
    print "FINAL:"  self
    return self
```

🔸 6. Subtle Behavior (Expert-Level)

⚠️ Optional Fields Missing

```python
class A BaseModel
    x int | None = None
```

In `after`:

- `self.x` exists (even if None)

In `before`:

- `"x"` may not exist in `data`

⚠ Extra Fields Handling

If model config allows extras:

```python
data "unknown"   # still present in before
```

But may disappear after parsing.

⚠ Validation Errors Aggregation

If multiple errors occur:

- Pydantic collects them
- BUT model validators often raise **single error**

👉 So structure messages clearly

🔸 7. Clean Architecture Approach

In production-level code:

✔ Use `before`

- input normalization
- backward compatibility
- schema evolution

✔ Use `after`

- business rules
- invariants

- derived logic

🔚 Final Mental Model

- `before` = **“fix the input”**
- `after` = **“enforce truth”**

If you want to go even deeper next, I can:

- Show **how model validators interact with FastAPI request lifecycle**
- Or give you **real buggy examples** and make you debug them (this is how you reach expert level fast)

dont just jump to code. explain me model validator, what is before and after, what is classmethod, what is cls, what is ConfigDict

Good—this is the right way to approach it. If the foundations are clear, the code becomes obvious instead of “magic.”

Let’s build this step by step.

🔹 What is a Model Validator (Conceptually)

A **model validator** is a hook that lets you **intercept and control how a model is created**.

Think of a Pydantic model like this:

[redacted]

A **model validator** lets you step into that process at specific moments.

👉 So instead of just *accepting whatever Pydantic builds*, you can:

- modify the input
- enforce rules across fields

- reject invalid combinations

🔹 The Two Phases: “before” vs “after”

This is the most important concept.

🟡 `mode="before"` → *Before Pydantic understands the data*

At this stage:

- Data is still **raw input**
- Usually a **dict**
- Types are NOT enforced yet

Mental picture:

```
```

Example intuition:

Input:

```JSON
"25"
```

In `before`:

- `"25"` is still a string
- You can convert it, clean it, reshape it

👉 This phase is about:

- cleaning
- normalizing
- restructuring

🔵 `mode="after"` → *After Pydantic builds the model*

At this stage:

- Data is already parsed
- You get a **model instance**
- Types are correct

Mental picture:

[redacted]

Now:

- `age` is already an `int`
- All fields are available

👉 This phase is about:

- enforcing rules
- validating relationships
- computing derived values

⚖️ Simple Analogy

Think of it like cooking:

- **before** → washing, cutting ingredients 🥕
- **after** → tasting the final dish 🍲

🔹 What is `@classmethod`?

This is **Python**, not Pydantic-specific.

A class method:

- belongs to the **class**, not an instance
- receives `cls` instead of `self`

Three method types in Python:

Type	First Argument	Meaning
Instance method	`self`	works on object
Class method	`cls`	works on class
Static method	none	utility

Why Pydantic uses `@classmethod` (in `before`)

In `mode="before"`:

- the model **does not exist yet**
- so there is no `self`

👉 That's why it uses:

```python
@classmethod
def validate cls  data
```

🔹 What is `cls`?

`cls` = the **class itself**

Example:

```python
class User
    pass
```

Inside a classmethod:

```python
cls == User
```

Why is `cls` useful?

You can:

- access class-level info
- create instances manually (advanced cases)
- support inheritance

🔹 What is `self` (in contrast)?

In `mode="after"`:

```python
def validate self
```

- `self` = actual model object
- already created by Pydantic

👉 So:

- `cls` → blueprint
- `self` → finished object

🔹 What is `ConfigDict`?

This is Pydantic's way to **configure model behavior**.

In Pydantic v2, instead of a `Config` class, we use:

```python
model_config = ConfigDict
```

Conceptually

`ConfigDict` answers:

👉 *"How should this model behave?"*

Not:

- what data is

But:

- how validation works
- how strict it is
- how it handles extras

Common things it controls

1. Extra fields

```python
model_config = ConfigDict extra="forbid"
```

- `"forbid"` → reject unknown fields

- `"allow"` → accept them
- `"ignore"` → silently drop

---

2. Strictness

```python
model_config = ConfigDict strict=True
```

- No automatic type coercion
- `"25"` → ❌ not converted to int

---

3. Validation behavior

```python
model_config = ConfigDict validate_assignment=True
```

- Re-validate when attributes change

🔹 How All These Fit Together

Let's connect everything:

```
[redacted]
```

And `ConfigDict` influences how strict or flexible this pipeline is.

🔹 Intuition You Should Walk Away With

- **model validator** = control point over full object
- **before** = raw data stage (use `cls`)

- **after** = final object stage (use `self`)
- **classmethod** = needed when object doesn't exist yet
- **cls** = class blueprint
- **ConfigDict** = behavior settings for validation system

If you're ready next, we can:

- walk through a **single example and trace it step-by-step through before → after**
- or I can give you **"predict the output" scenarios** (these sharpen understanding very fast)

Walk through a single example and trace it step-by-step through before → after

Perfect—this is where things really "click."
We'll take **one example**, and literally trace what happens at each stage.

🔹 The Example (Keep it small but realistic)

We'll imagine an API that takes:

```JSON
  " john doe "
"25"
 10
5
```

And we define a model that:

- cleans the name
- converts age
- validates that `start < end`

The Model (just for reference)

Python

```python
from pydantic import BaseModel  model_validator  ConfigDict

class User BaseModel
    name  str
    age  int
    start  int
    end  int

    model_config = ConfigDict extra="forbid"

    @model_validator mode="before"
    @classmethod
    def before_stage cls  data
        print "1. BEFORE VALIDATOR:"  data

        # clean name
        if "name" in data
            data "name"  = data "name" .          .

        return data

    @model_validator mode="after"
    def after_stage self
        print "3. AFTER VALIDATOR:"  self

        if self.         >= self.
            raise ValueError "start must be less than end"

        return self
```

🔥 Now Let's Trace Execution Step-by-Step

🟡 STEP 0: Raw Input

Python

```python
"name"  "  john doe  "
"age"  "25"
"start"  10
"end"  5
```

👉 This is what your API / user sends.

🟡 STEP 1: `model_validator(mode="before")`

This runs **first**.

```
```

Important observations:

- `age` is still `"25"` (string)
- nothing is parsed yet
- this is raw data

What we do here:

```python
data "name"  = data "name" .        .
```

So now:

```python
"name"  "John Doe"
"age"  "25"
"start"  10
"end"  5
```

👉 We return this modified dict

🔵 STEP 2: Pydantic Internal Parsing

Now Pydantic takes over.

It:

- converts types
- validates fields

So:

```python
"age" "25" → age: 25 (int)
```

Now internally we have:

```python
User
  name="John Doe"
  age=25
  start=10
  end=5
```

🔵 STEP 3: `model_validator(mode="after")`

Now this runs:

```
```

Key observations:

- This is a **real object**
- types are correct
- we use `self`, not `cls`

Now the logic runs:

```python
if self.        >= self.
    raise ValueError "start must be less than end"
```

Here:

```python
10 >= 5 → True
```

👉 So it raises an error

🔴 STEP 4: Final Result

Pydantic stops and returns:

```
[redacted]
```

🔁 What If Input Was Correct?

Let's change input:

```JSON
"  john doe  "
"25"
5
10
```

Flow:

STEP 1 (before):

```Python
"name"    "John Doe"
```

STEP 2 (parsing):

```Python
"25" → 25
```

STEP 3 (after):

```python
5 < 10 → OK
```

✅ Final Model:

```python
User
  name="John Doe"
  age=25
  start=5
  end=10
```

🔥 Key Insights You Should Notice

1. Data evolves across stages

---

2. `before` ≠ safe for logic

This would be WRONG:

```python
if data "start"  >= data "end"   # ❌ risky
```

👉 Because:

- values might be strings
- fields might be missing

---

3. `after` is safest for rules

Because:

- everything exists
- everything is typed

4. You control transformation

Without `before`:

```python
"name" " john doe "
```

With `before`:

```python
"name" "John Doe"
```

👉 That's silent normalization (very common in APIs)

---

5. Error timing matters

- Errors in `before` → early rejection
- Errors in `after` → logical/business rejection

🔚 Final Mental Model (Now Solid)

[redacted]

If you want next, I can:

- give you **tricky edge cases** (like missing fields, optional fields, nested models)
- or make you **predict outputs step-by-step** (this is how you become very fast at debugging)